ILCSS — Iterative Longest Common SubStrings Similarity: A String Similarity Measure for Multi-Part Personal Name Matching

Raoul Sotero Calao*

2026

Preprint: doi:10.5281/zenodo.19462182

Abstract

We present ILCSS (Iterative Longest Common SubStrings Similarity), a string similarity measure specifically designed for approximate matching of multi-part personal names from heterogeneous sources. The problem arises when the same individual's name appears with its components in different orders or with a different number of parts across administrative databases, criminal intelligence systems, or bibliographic records — a case that defeats character-level measures such as Levenshtein distance and Jaro-Winkler. ILCSS iteratively extracts the longest common substrings shared by two strings, consuming matched blocks at each step and applying a positional penalty that attenuates with block length, thereby rewarding component-level agreement while retaining sensitivity to ordering conventions. We prove that ILCSS is a similarity coefficient — bounded, reflexive, and conditionally symmetric — but not a metric distance. A synthetic benchmark of 295 name pairs drawn from five naming traditions (Arabic, Hispanic, Portuguese, Italian, French/German), covering all permutations, component-drop, and insertion variants, shows that ILCSS achieves a same-person/different-person score gap of 0.729, compared to 0.262–0.361 for competing measures, with F1 = 0.998 at an optimal threshold of 0.40. Evaluation on two real-world record-linkage datasets (DBLP-ACM and FEBRL) confirms the advantage, with F1 = 0.994 and 0.989 respectively. Reference implementations in Perl, Crystal, and Python are provided.

1. Problem Statement

Matching personal names across heterogeneous databases is a central challenge in record linkage and entity resolution [18, 19]. The difficulty goes beyond simple typographic variation: names from different sources — civil registries, immigration records, administrative databases, transliterated documents — frequently differ in the **order** and **number** of their constituent parts, while the parts themselves remain identical or near-identical.

Consider the following pairs, each referring to the same individual:

Source A	Source B	Variation type
`mario rossi`	`rossi mario`	Two-part transposition
`mario rossi`	`mario giuseppe rossi`	Middle name insertion
`abdullah abdel aziz`	`aziz abdullah abdel`	Three-part permutation

*Chief Commissioner, Italian National Police - Central Anticrime Directorate
(raoulsotero.calao@poliziadistato.it; rscalao@gmail.com)

Source A	Source B	Variation type
`abdullah abdel aziz`	`abdel aziz abdullah`	Three-part permutation
`maria rosa garcia lopez`	`garcia lopez maria rosa`	Block transposition

This problem is particularly acute for Arabic names, which commonly consist of a chain of three or more components — a given name followed by the father's name, grandfather's name, and optionally a family name — recorded in different orders across different administrative traditions. It also arises systematically with Hispanic double-surname naming conventions, Portuguese and Brazilian multi-part surnames, and historical records where name order was not standardized.

A practically important domain where this challenge is especially severe is **law enforcement and criminal intelligence analysis**. Investigations targeting organized crime networks and domestic or international terrorism routinely require cross-referencing personal name data across heterogeneous sources — criminal records, immigration and border control systems, intelligence reports, court documents, watchlists, and international law-enforcement information exchanges such as Interpol notices or Europol databases — each of which may follow different national conventions for name ordering and may record a different subset of an individual's name components. A subject who appears as `"ali hassan abdel rahman"` in one system may be recorded as `"abdel rahman ali"` or simply `"hassan ali"` in another, and standard character-level similarity measures will fail to identify these as the same individual. The consequences of a missed match — or of a spurious one — in this context are operationally significant, making reliable approximate name matching a genuine requirement rather than a convenience.

Existing string similarity measures handle these cases poorly. **Levenshtein edit distance** [3] treats each character substitution, insertion, and deletion as a unit cost: transposing two long name parts requires rewriting almost the entire string, producing a large distance even when all parts are identical. **Jaro-Winkler** [15, 16] explicitly rewards agreement at the beginning of the string and applies a positional window for character matching; for `"abdullah abdel aziz"` vs `"aziz abdullah abdel"`, neither the prefix nor the character-position assumptions hold, and the score is correspondingly low. **Ratcliff/Obershelp** [2] finds the longest common block recursively and computes similarity as twice the matched characters over total characters; it correctly identifies the common blocks but has no mechanism to distinguish a same-person pair — where all parts are present, merely reordered — from a different-person pair where only some parts coincide.

Token-based methods (Jaccard on word tokens, sorted-token similarity) address the ordering problem by treating the name as a bag of words, but discard all information about the relative positioning of parts, which carries genuine signal when sources are expected to use similar conventions.

---

2. The ILCSS Approach

ILCSS (Iterative Longest Common SubStrings) measures the similarity between two strings in the range [0, 1]. It treats the full name — including all parts and spaces — as a single character string. Rather than requiring a single global alignment, it **iterates**: at each step it identifies all longest common substrings shared by the two strings, "consumes" them by replacing them with placeholders (so they are not re-matched), and adds their contribution to the score. The process repeats until no further block of length ≥ 3 remains.

The key design element is the **positional penalty**: each matched block contributes to the score in proportion to its length, but the contribution is attenuated if the block appears at very different positions in the two strings. The attenuation itself decreases for longer blocks — a long block matching across a large positional gap is still strong evidence of identity. This balances two competing signals: same-part content (matched block length) and same-ordering convention (positional agreement).

For the pair `"abdullah abdel aziz"` / `"aziz abdullah abdel"`:

1. First iteration finds `"abdullah"` (length 8) in both strings, at positions 0 and 5 respectively. The positional distance is 5 out of L = 19; the penalty is moderate and attenuated by the block's relative length ($8/19 \approx 0.42$). Block score ≈ 7.1.
2. After consuming `"abdullah"`, finds `"abdel"` (length 5) at positions 9 and 12 in the modified strings. Similar attenuation. Block score ≈ 4.3.
3. After consuming `"abdel"`, finds `"aziz"` (length 4) at positions 15 and 0. Larger positional gap, but again attenuated. Block score ≈ 2.9.
4. No further blocks of length ≥ 3. Total score ≈ 14.3. Similarity $\approx 14.3 / 19 \approx$ **0.75**.

The same pair scored by Levenshtein (normalized) gives approximately 0.37; Jaro-Winkler gives approximately 0.55. ILCSS correctly identifies the pair as highly similar.

---

3. Pseudocode

```
function ilcss_sim(s1, s2):
    s1 := remove_spaces(s1)
    s2 := remove_spaces(s2)
    L := max(length(s1), length(s2))      # reference length for normalization
    total_score := 0

    repeat:
        blocks := longest_common_substrings(s1, s2)
                  # returns all common substrings of maximum length
        min_len := length(blocks[0])      # all blocks have the same length

        for each block in blocks:
            b := length(block)
            if b < 3: skip to next       # blocks shorter than a trigram are ignored

            p1 := position of block in s1
            p2 := position of block in s2

            # "consume" the block so it is not matched in subsequent iterations
            s1 := replace(s1, p1, b, "{" * b)
            s2 := replace(s2, p2, b, "}" * b)

            # penalty based on positional distance between the two blocks
            dist        := |p2 - p1|
            r_dist      := dist / L                # relative distance
```

```
            r_len       := b / L                    # relative block length
            r_penalty   := r_dist * (1 - r_len)   # longer blocks reduce the penalty
            total_score := total_score + b * (1 - r_penalty)

    until min_len <= 2

    return total_score / L               # similarity normalized in [0, 1]

function longest_common_substrings(s1, s2):
    # space-optimized DP [3, 5]: only two columns kept in memory
    prev[0..m] := 0,  curr[0..m] := 0    # m = length of s2
    best := 0
    results := []

    for i from 1 to length(s1):
        for j from 1 to length(s2):
            if s1[i-1] == s2[j-1]:            # 0-indexed character access
                curr[j] := prev[j-1] + 1
                if curr[j] > best:
                    best := curr[j]
                    results := [ s1[i-best .. i] ]       # new list with this one only
                else if curr[j] == best:
                    results.append( s1[i-best .. i] )   # tie: add to the list
            else:
                curr[j] := 0
        swap prev and curr
        reset curr to 0

    return results
```

---

4. Relationship to prior work

4.1 Record linkage and personal name matching

The probabilistic framework for record linkage was established by Fellegi & Sunter [18], who formalized the problem of determining whether two records from different sources refer to the same entity. Personal name matching is among the most studied subtasks within this framework [19]. Surveys of name-matching techniques — notably Christen [20] and Lait & Randell [21] — consistently identify the core difficulty as the mismatch between the diversity of real-world name representations and the assumptions of character-level distance measures.

The particular challenge of **multi-part names with variable component order** has received growing attention in several specific domains. For Spanish names, Ruiz-Pérez et al. [22] documented that between 48% and 69% of Spanish authors appear under multiple name forms across major bibliographic databases, largely because of inconsistent handling of the double-surname convention; no dedicated matching algorithm for this problem is available in the literature. For genealogical records, Wilson [23] explicitly addresses first-name/last-name transposition and proposes combined-field

indexing as a mitigation, without resolving the general case of arbitrary component permutations.

Arabic personal names pose the most severe instance of this problem. Arabic naming tradition commonly chains three or more components — given name, father's name, grandfather's name, and optionally a tribal or family name — and different administrative systems record these chains in different orders. El-Shishtawy [24] addresses Arabic name-based database linkage directly, proposing a rule-based system for name normalization prior to matching. Freeman, Condon & Ackerman [25] extend edit distance with Arabic-English character equivalency classes to handle transliteration variants, but neither work addresses the component-reordering problem algorithmically. Hassanat & Altarawneh [26], working from a corpus of 9.9 million Jordanian citizen names, systematically catalogue the orthographic and phonological spelling variants that arise in Arabic name databases — character substitutions, Hamzah deletion, definite-article attachment, feminization suffixes, and encoding-driven substitutions from legacy 7-bit systems — and propose a rule-and-dictionary approach for normalisation. Their work addresses within-token variation exclusively; the across-token component-reordering problem (the same person's name components appearing in different orders across records) is outside their scope and is not addressed by any of the standard string similarity measures surveyed in the literature.

4.2 String similarity measures

The Jaro comparator [15] and its Winkler extension [16], developed for the U.S. Census name-matching problem, explicitly reward prefix agreement and apply a character-matching window proportional to string length. These design choices are well-suited for typographic variants of single-word names but work against multi-part name matching: for `"abdullah abdel aziz"` vs `"aziz abdullah abdel"`, prefix agreement is zero and the positional window misaligns most character matches.

Token-based approaches — Jaccard coefficient on word tokens, sorted-token Jaro-Winkler (Soft-TFIDF, as evaluated in Cohen, Ravikumar & Fienberg [17]) — handle component reordering by treating the name as a bag of words. This correctly addresses full permutations but discards all positional information, so a name with all correct parts in the wrong order scores identically to one where a part has been randomly substituted. For a matching task where both same-person and similar-but-different-person pairs are present, this loss of positional signal is a liability.

The idea of iteratively extracting the longest common substring was described as early as 1967 by Alberga [1], who attributes it to unpublished work by Baskin and Selfridge at IBM Research — cited only as private communications and never published independently. The closest published algorithm sharing this iterative structure is the Ratcliff/Obershelp method [2] (1988), which recursively finds the longest common substring and then recurses on the flanking portions, computing similarity as twice the number of matched characters divided by total characters. ILCSS differs from Ratcliff/Obershelp in two key respects: it applies a **positional penalty** (blocks found at very different positions in the two strings contribute less to the score, but less so for longer blocks), and it processes all blocks of equal maximum length at each iteration rather than a single one.

The underlying dynamic programming recurrence for the longest common substring is a standard result in string algorithmics [3, 4]. The space optimization that keeps only two columns of the DP table in memory — reducing space from O(nm) to O(m) — follows the approach described by Hirschberg [5] for the related longest common subsequence problem. For a broad survey of string similarity and approximate matching techniques, see Navarro [6].

5. Key properties

Minimum block length. Blocks shorter than 3 characters are discarded. The minimum length of 3 reflects the well-established role of trigrams as the smallest orthographic unit with sufficient discriminative power for string matching. Blocks of length 1 or 2 match too frequently by chance: over a 26-character Latin alphabet, there are only $26^2 = 676$ distinct bigrams, many of which (e.g., *th*, *he*, *in*) occur with very high frequency in natural-language text, producing a high rate of spurious matches. Trigrams yield $26^3 = 17{,}576$ distinct types with a considerably flatter distribution, providing meaningful selectivity without the storage and computational overhead of longer n-grams [7]. This choice is consistent with the information-theoretic argument underlying Ukkonen's q-gram filter [8]: smaller values of q produce weaker filters, allowing more candidate pairs to pass through. Empirically, Willett [9] directly compared digram and trigram encoding on the Cranfield test collection and found that *"trigram encoding of words performs noticeably better than the use of digrams"*; Angell, Freund and Willett [10] subsequently adopted trigrams as the operative unit in their canonical trigram-based spelling correction system, achieving over 90% correct identification against a 64,636-word dictionary.

Iteration. At each cycle the matched blocks are masked, so the next iteration searches for matches in the remaining material, capturing all shared name parts regardless of their number and order.

Positional penalty. A common block is worth less if it appears at very different positions in the two strings. The formula `r_dist × (1 - r_len)` attenuates the penalty for longer blocks: a long name part is strong evidence of identity even if positionally displaced. This means that fully reordered names (all parts present, different order) score high, while partially overlapping names (only some parts in common) score proportionally lower.

Normalization. Dividing by `L` (the length of the longer string) guarantees a result in [0, 1], where 1 means identical strings (ignoring spaces) and 0 means no common substring of length ≥ 3. Using the longer string as the reference penalizes cases where one name is a proper subset of the other (e.g., `"mario rossi"` vs `"mario giuseppe rossi"`).

Complexity. Each iteration runs the DP in O(n·m); the number of iterations is at most O(min(n,m)/3), so the overall complexity is O(n·m·min(n,m)).

---

6. Formal analysis of metric properties

ILCSS computes a **similarity measure** — a function mapping pairs of strings to [0, 1] — not a distance. Following the axiomatic framework of Santini & Jain [11] and the survey by Lesot, Rifqi & Benhadda [12], we examine each relevant property in turn.

Boundedness and non-negativity. Since `r_dist` $\in$ `[0,1]` and `r_len` $\in$ `[0,1]`, we have `r_penalty = r_dist × (1 - r_len)` $\in$ `[0,1]`, so `block_score = b × (1 - r_penalty)` $\geq$ `0`. The sum of all matched block lengths cannot exceed `min(|s1|, |s2|)` $\leq$ `L`, hence `total_score / L` $\leq$ `1`. Therefore $\mathbf{0 \leq sim(s1, s2) \leq 1}$. ■

Reflexivity. If `s1 = s2` (after whitespace removal), the first call to `longest_common_substrings` returns the whole string as the unique block, with `dist = 0`, `r_penalty = 0`, and `block_score = |s1|`. Thus `total_score / L = 1`. Conversely, `sim = 1` implies matched length equals `L`, which — given the no-overlap constraint — requires `s1 = s2`. Hence $\mathbf{sim(s1, s2) = 1 \iff s1 = s2}$ (modulo spaces). ■

Symmetry — conditional. The positional penalty `|p2 - p1| / L` and the normalization by `L = max(|s1|, |s2|)` are both symmetric under exchange of s1 and s2. However, when `longest_common_substrings` returns multiple blocks of equal maximum length, the tie-breaking order depends on the loop structure (outer loop over s1), and subsequent `index` calls select the first occurrence in the modified string. Swapping s1 and s2 may therefore produce a different sequence of consumed blocks across iterations and hence a different score. **Symmetry holds when the LCS is unique at each iteration; it is not guaranteed in the presence of ties.**

Triangle inequality — does not hold in general. Converting ILCSS to a distance via `d(s1, s2) = 1 - sim(s1, s2)`, the triangle inequality `d(s1, s3)` $\leq$ `d(s1, s2) + d(s2, s3)` is not satisfied in general. This follows from the result established by Kondrak [13] that normalized LCS-based similarity measures lose the triangle inequality when converted to distance: normalization by string length introduces a non-linearity that invalidates the inequality. The same phenomenon is documented for normalized edit distance by Li & Liu [14], who propose a corrected normalization formula that restores the metric property; an analogous correction for ILCSS is left as future work.

Positional sensitivity as a deliberate design choice. The positional penalty places ILCSS in the family of *positionally sensitive* string comparators, alongside the Jaro comparator [15] and its Jaro-Winkler extension [16], both of which are also non-metrics. The underlying intuition — that common substrings aligned at similar positions provide stronger evidence of similarity than displaced ones — is empirically validated for name-matching and entity resolution tasks in Cohen, Ravikumar & Fienberg [17].

Summary.

Property	Holds?	Notes
$0 \leq \text{sim} \leq 1$	✓	Proven above
$\text{sim}(s, s) = 1$	✓	Proven above
$\text{sim} = 1 \iff s1 = s2$	✓	Proven above
Symmetry	Conditional	Holds unless tie-breaking produces different block sequences
Triangle inequality (1 – sim)	×	Follows from Kondrak [13]; normalization destroys it

ILCSS is therefore a **similarity coefficient** in the sense of Lesot et al. [12] — satisfying reflexivity and boundedness, with conditional symmetry, but not constituting a metric distance. This places it in the same class as Jaro-Winkler and other positionally sensitive comparators used in record linkage.

---

7. Experimental evaluation

7.1 Synthetic dataset

To evaluate ILCSS against competing measures on the problem class for which it is designed, we constructed a synthetic benchmark of 295 string pairs covering the variation types described in Section 1. Starting from 22 base names drawn from five naming traditions — Arabic (3- and 4-part

chains), Hispanic (2-, 3-, and 4-part), Portuguese (2- and 3-part), and European 2-part (Italian, French, German) — we generated:

- **All non-identity permutations** of each name's components (2 permutations for 2-part, 5 for 3-part, 23 for 4-part);
- **Component-drop variants**: removal of the first, middle, or last part from 3- and 4-part names;
- **Insertion variants** for 2-part names: addition of a middle component or a trailing suffix;
- **Negative (different-person) pairs**: each base name paired with three names from different entries in the set, spanning cross-cultural combinations.

The dataset contains 22 identity pairs, 207 same-person variant pairs, and 66 different-person negative pairs (273 pairs excluding identities). Ground truth is exact by construction. All names are in romanized Latin script; Arabic-script orthographic variation is not tested here (see Section 7.4).

7.2 Algorithms compared

Four algorithms were evaluated on each pair:

Algorithm	Description
ILCSS	This work; threshold 3, positional penalty as described in Section 3
NormLev	1 – edit_distance(s1, s2) / max(
Jaro-Winkler	Standard implementation (prefix scale p = 0.1)
Ratcliff/Obershelp	Recursive LCS; similarity = 2 × matched / (

7.3 Results

Mean similarity scores by variation class (identity pairs excluded from aggregates):

Variation class	ILCSS	NormLev	Jaro-Winkler	Ratcliff/Ob	N
Identity	1.000	1.000	1.000	1.000	22
2-part swap	0.757	0.110	0.574	0.517	9
3-part permutations (all)	0.788	0.402	0.789	0.640	45
4-part permutations (all)	0.781	0.433	0.813	0.639	92
Drop one component	0.654	0.683	0.877	0.809	43
Insert / append	0.823	0.796	0.949	0.886	18
Same person (all variants)	**0.759**	**0.496**	**0.823**	**0.691**	**207**
Different person (negative)	**0.030**	**0.231**	**0.561**	**0.330**	**66**

Variation class	ILCSS	NormLev	Jaro-Winkler	Ratcliff/Ob	N
Gap Δ (same − different)	**0.729**	**0.265**	**0.262**	**0.361**	—

The most diagnostic figure is the gap between the mean scores for same-person pairs and different-person pairs. ILCSS achieves $\Delta = 0.729$, more than twice the gap of Jaro-Winkler ($\Delta = 0.262$) and Normalized Levenshtein ($\Delta = 0.265$), and substantially better than Ratcliff/Obershelp ($\Delta = 0.361$). The large gap reflects two complementary behaviors: ILCSS scores reordered same-person pairs high (≥ 0.75 across all permutation types) while scoring unrelated name pairs close to zero (mean 0.030), because the trigram threshold suppresses the short chance matches between structurally unrelated names.

Jaro-Winkler achieves the highest mean score on same-person variants (0.823) but also the highest mean on different-person pairs (0.561), reducing the discriminative margin to the point where a fixed decision threshold cannot separate the two classes reliably. Normalized Levenshtein degrades sharply on permutations (0.110 for 2-part swaps) because even identical character sets reordered within a string entail a near-total character-level rewrite.

Optimal decision threshold and F1 performance:

Algorithm	Optimal θ	Precision	Recall	F1	Accuracy
ILCSS	**0.40**	**0.995**	**1.000**	**0.998**	**0.996**
Ratcliff/Obershelp	0.45	0.948	0.971	0.959	0.938
Jaro-Winkler	0.65	0.960	0.928	0.944	0.916
NormLev	0.25	0.884	0.879	0.881	0.821

At threshold $\theta = 0.40$, ILCSS produces a single false negative (one same-person pair scored just below the threshold) and zero false positives, for F1 = 0.998 and accuracy = 0.996. The precision curve for ILCSS reaches 1.000 at $\theta = 0.40$ and remains at 1.000 for all higher thresholds, because no different-person pair scores above 0.40 in this dataset. The recall curve remains above 0.96 down to $\theta = 0.55$, indicating robustness to threshold choice over a wide range.

Jaro-Winkler requires a substantially higher threshold (0.65) to achieve acceptable precision, because many different-person pairs score between 0.50 and 0.65; at the same threshold, recall drops to 0.928, meaning 7% of same-person pairs are missed. Normalized Levenshtein achieves its best F1 at the low threshold of 0.25, reflecting the algorithm's inability to score permutation pairs above 0.50 — a correct classification regime but with poor separation and correspondingly lower accuracy.

7.4 Discussion

Component drop. For pairs where one name is a proper subset of another (e.g., `"juan garcia lopez"` / `"juan garcia"`), ILCSS gives a middle-range score (mean 0.664) rather than a high one. This is the intended behavior: normalization by the longer string penalizes missing components, preventing spurious high-confidence matches when a name has been truncated. In a real record-linkage system, such pairs would fall in a decision region requiring additional evidence, not automatic acceptance.

Positional penalty in context. The penalty attenuates but does not eliminate the positional signal. A fully reversed 3-part name scores approximately 0.69–0.85, depending on component lengths; a fully reversed 2-part name scores approximately 0.75. These scores are well above the optimal threshold of 0.40, confirming that the penalty does not harm recall for the reordering case.

Jaro-Winkler on multi-part names. Jaro-Winkler performs surprisingly well on 3- and 4-part permutations (mean 0.771–0.820), but the high scores on unrelated names (mean 0.561) make discrimination difficult. The issue is structural: Jaro-Winkler's character-matching window scales with string length, so for long multi-part names it finds many character-level coincidences even between unrelated names sharing common suffixes (e.g., `-ez`, `-ia`, `-er`). ILCSS avoids this because the trigram threshold requires at least three consecutive matching characters, which is a much stronger condition than isolated character matches within a window.

Limitations of the synthetic evaluation. The dataset pairs are generated by exact structural transformations; real-world name pairs additionally exhibit typographic errors, transliteration variants, partial abbreviations (initials), and combinations of multiple variation types. The component-drop and insert tests cover only single-component changes; chains of two or more simultaneous variations are not represented. Furthermore, all names in the dataset are in romanized Latin script; Arabic-script orthographic variation — as catalogued for example by Hassanat & Altarawneh [26] — is a distinct and complementary challenge not addressed here. An evaluation on a real annotated corpus of name pairs (see Section 8) would be required to confirm these findings in operational conditions.

---

8. Evaluation on real-world data

To complement the controlled synthetic evaluation (Section 7), we tested ILCSS on two publicly available record-linkage benchmarks that contain real name variation, albeit of a different character than the multi-part reordering problem.

8.1 DBLP-ACM author names

The DBLP-ACM dataset (Leipzig Benchmark, CC BY 4.0 [27]) contains 2,224 matched pairs of publication records drawn from the DBLP and ACM digital libraries, with a manually verified perfect mapping as ground truth. For each matched pair we extracted the author name strings, normalised to lower-case with initial letters removed, and compared the DBLP author string against the ACM author string as a single character sequence; 5 pairs were dropped because one record had an empty author field, leaving 2,219 usable matched pairs. Non-matched pairs (N = 222) were sampled by misaligning indices across the matched set.

The dataset is not designed for within-name component reordering: it contains full author lists (multiple names separated by commas), and variation arises from abbreviation styles, typographic errors, and inconsistent author-list ordering across the two libraries. Nonetheless, the matched pairs exhibit the same structural challenge ILCSS addresses: long strings with identical content in different positional arrangements.

Measure	Same person (N=2219)	Different (N=222)	Gap	Best thr.	F1	Accuracy
ILCSS	**0.783**	**0.025**	**0.758**	**0.15**	**0.994**	**0.990**
Ratcliff/Ob	0.716	0.223	0.493	0.30	0.984	0.971
Jaro-Winkler	0.854	0.548	0.306	0.60	0.971	0.948
NormLev	0.569	0.185	0.384	0.05	0.950	0.904

ILCSS achieves the largest gap between matched and non-matched pairs (0.758 vs. 0.306–0.493 for the other algorithms), and reaches F1 = 0.994 at threshold 0.15 — lower than the synthetic-data optimum of 0.40, reflecting that some matched author strings differ substantially in length due to abbreviation. Jaro-Winkler's gap is the smallest (0.306) because its character-window mechanism matches many characters between unrelated author lists that share common name fragments.

We identified a small subset of 1 pair (out of 2,219) where the matched records contain the exact same author-name tokens in a different order — confirming that this variation type exists in the real data, though at low frequency in this particular benchmark. For that pair: ILCSS = 0.780, NormLev = 0.548, Jaro-Winkler = 0.865, Ratcliff/Ob = 0.715.

8.2 FEBRL given-name / surname swap

The FEBRL dataset1 [28] is a synthetic record-linkage benchmark with 500 matched pairs generated with controlled noise: field corruption, typographic errors, and — relevant to ILCSS — a given-name/surname field swap applied to approximately 7% of matched pairs. We concatenated the `given_name` and `surname` fields into a single name string before comparison.

Measure	Same person (N=500)	Different (N=100)	Gap	Best thr.	F1	Accuracy
ILCSS	**0.825**	**0.010**	**0.816**	**0.15**	**0.989**	**0.982**
Ratcliff/Ob	0.874	0.268	0.606	0.40	0.987	0.978
Jaro-Winkler	0.905	0.512	0.393	0.65	0.945	0.913
NormLev	0.811	0.187	0.625	0.30	0.944	0.910

For the 35 swapped pairs specifically (same tokens, first/last position exchanged), ILCSS scores mean 0.758, maintaining good recall at threshold 0.15. NormLev degrades sharply to 0.171 on these pairs — consistent with the synthetic results — while Jaro-Winkler (0.621) and Ratcliff/Ob (0.531) both score below the 0.65 and 0.40 thresholds respectively, meaning they would misclassify a fraction of the swapped pairs as non-matches.

8.3 Discussion

Across both real datasets, ILCSS consistently achieves the largest gap between matched and non-matched name pairs. The F1 optima on real data (0.15 for DBLP-ACM and FEBRL) are lower than on the synthetic data (0.40), because real matched records contain noise that reduces scores for

genuine matches. This suggests that a threshold of approximately 0.40 is appropriate when names are relatively clean (administrative databases with standardized input), while a threshold of 0.15–0.25 is more appropriate when names are subject to abbreviation, partial records, or transcription errors.

We note a gap in the available public benchmarks: no dataset with ground truth currently covers the full problem stated in Section 1 — namely, multi-part personal names from culturally diverse sources where the same individual's components appear in systematically different orders. The LREC 2008 dataset (Arehart & Miller) [29] was specifically designed for this problem and includes Arabic names with segmentation variation, but has not been publicly released. Filling this benchmark gap is an important open task for the record-linkage community.

References

[1] C. N. Alberga, "String Similarity and Misspellings," *Communications of the ACM*, Vol. 10, No. 5, May 1967, pp. 302–313. DOI: 10.1145/363282.363371. *(Cites H. Baskin and L. D. Selfridge, IBM Research Center, as private communications [refs. 1 and 3 therein] — the iterative longest-common-substring method was never independently published by either author.)*

[2] J. W. Ratcliff and D. E. Metzener, "Pattern Matching: The Gestalt Approach," *Dr. Dobb's Journal*, Vol. 13, No. 7, July 1988, p. 46.

[3] R. A. Wagner and M. J. Fischer, "The String-to-String Correction Problem," *Journal of the ACM*, Vol. 21, No. 1, January 1974, pp. 168–173. DOI: 10.1145/321796.321811.

[4] D. Gusfield, *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*, Cambridge University Press, 1997. ISBN: 978-0-521-58519-4.

[5] D. S. Hirschberg, "A Linear Space Algorithm for Computing Maximal Common Subsequences," *Communications of the ACM*, Vol. 18, No. 6, June 1975, pp. 341–343. DOI: 10.1145/360825.360861.

[6] G. Navarro, "A Guided Tour to Approximate String Matching," *ACM Computing Surveys*, Vol. 33, No. 1, March 2001, pp. 31–88. DOI: 10.1145/375360.375365.

[7] R. Cox, "Regular Expression Matching with a Trigram Index," 2012. https://swtch.com/~rsc/regexp/regexp4.html

[8] E. Ukkonen, "Approximate String-Matching with q-Grams and Maximal Matches," *Theoretical Computer Science*, Vol. 92, No. 1, 1992, pp. 191–211. DOI: 10.1016/0304-3975(92)90143-4.

[9] P. Willett, "Document Retrieval Experiments Using Indexing Vocabularies of Varying Size. II. Hashing, Truncation, Digram and Trigram Encoding of Index Terms," *Journal of Documentation*, Vol. 35, No. 4, 1979, pp. 296–305. DOI: 10.1108/eb026684.

[10] R. C. Angell, G. E. Freund, P. Willett, "Automatic Spelling Correction Using a Trigram Similarity Measure," *Information Processing & Management*, Vol. 19, No. 4, 1983, pp. 255–261. DOI: 10.1016/0306-4573(83)90022-5.

[11] S. Santini and R. Jain, "Similarity Measures," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 21, No. 9, September 1999, pp. 871–883. DOI: 10.1109/34.790428.

[12] M.-J. Lesot, M. Rifqi, H. Benhadda, "Similarity Measures for Binary and Numerical Data: A Survey," *International Journal of Knowledge Engineering and Soft Data Paradigms*, Vol. 1, No. 1, 2009, pp. 63–84. DOI: 10.1504/IJKESDP.2009.021985.

[13] G. Kondrak, "N-Gram Similarity and Distance," in *String Processing and Information Retrieval (SPIRE 2005)*, LNCS vol. 3772, Springer, 2005, pp. 115–126. DOI: 10.1007/11575832_13.

[14] Li Yujian and Liu Bo, "A Normalized Levenshtein Distance Metric," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 29, No. 6, June 2007, pp. 1091–1095. DOI: 10.1109/TPAMI.2007.1078.

[15] M. A. Jaro, "Advances in Record-Linkage Methodology as Applied to Matching the 1985 Census of Tampa, Florida," *Journal of the American Statistical Association*, Vol. 84, No. 406, June 1989, pp. 414–420. DOI: 10.1080/01621459.1989.10478785.

[16] W. E. Winkler, "String Comparator Metrics and Enhanced Decision Rules in the Fellegi-Sunter Model of Record Linkage," *Proceedings of the Section on Survey Research Methods*, American Statistical Association, 1990, pp. 354–359. ERIC: ED325505.

[17] W. W. Cohen, P. Ravikumar, S. E. Fienberg, "A Comparison of String Distance Metrics for Name-Matching Tasks," *Proceedings of the IJCAI-2003 Workshop on Information Integration on the Web (IIWeb-2003)*, 2003, pp. 73–78.

[18] I. P. Fellegi and A. B. Sunter, "A Theory for Record Linkage," *Journal of the American Statistical Association*, Vol. 64, No. 328, 1969, pp. 1183–1210. DOI: 10.1080/01621459.1969.10501049.

[19] P. Christen, *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*, Springer, 2012. ISBN: 978-3-642-31163-5. DOI: 10.1007/978-3-642-31164-2.

[20] P. Christen, "A Comparison of Personal Name Matching: Techniques and Practical Issues," *Proceedings of the Sixth IEEE International Conference on Data Mining Workshops (ICDMW 2006)*, 2006, pp. 290–294. DOI: 10.1109/ICDMW.2006.2.

[21] A. J. Lait and B. Randell, "An Assessment of Name Matching Algorithms," Technical Report No. 550, Department of Computing Science, University of Newcastle upon Tyne, 1996.

[22] R. Ruiz-Pérez, E. Delgado López-Cózar, E. Jiménez-Contreras, "Spanish Personal Name Variations in National and International Biomedical Databases: Implications for Information Retrieval and Bibliometric Studies," *Journal of the Medical Library Association*, Vol. 90, No. 4, 2002, pp. 411–430. PMCID: PMC128958.

[23] D. R. Wilson, "Name Standardization for Genealogical Record Linkage," *Proceedings of the Family History Technology Workshop (FHTW 2005)*, Brigham Young University, 2005.

[24] T. El-Shishtawy, "Linking Databases using Matched Arabic Names," *International Journal of Computational Linguistics & Chinese Language Processing*, Vol. 19, No. 1, 2014, pp. 33–54. ACL: O14-2003.

[25] A. Freeman, S. Condon, C. Ackerman, "Cross Linguistic Name Matching in English and Arabic," *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, 2006, pp. 471–478. ACL: N06-1060.

[26] A. B. A. Hassanat and G. A. Altarawneh, "Rule-and Dictionary-based Solution for Variations in Written Arabic Names in Social Networks, Big Data, Accounting Systems and Large Databases," *Research Journal of Applied Sciences, Engineering and Technology*, Vol. 8, No. 14, 2014, pp. 1630–1638. arXiv: 1502.05441.

[27] E. Köpcke, A. Thor, E. Rahm, "Evaluation of Entity Resolution Approaches on Real-World Match Problems," *Proceedings of the VLDB Endowment*, Vol. 3, No. 1, 2010, pp. 484–493. Dataset: DBLP-

ACM, available at https://dbs.uni-leipzig.de/research/projects/object_matching/benchmark_datasets_for_entity_
(CC BY 4.0).

[28] P. Christen and K. Goiser, "Quality and Complexity Measures for Data Linkage and Deduplication," in F. Guillet and H. Hamilton (eds.), *Quality Measures in Data Mining*, Springer, 2007, pp. 127–151. FEBRL dataset available via Python Record Linkage Toolkit (https://github.com/J535D165/recordlinkage, BSD-3-Clause).

[29] M. Arehart and K. J. Miller, "A Ground Truth Dataset for Matching Culturally Diverse Romanized Person Names," *Proceedings of LREC 2008*, Marrakech, 2008, pp. 2064–2069.